

Spam_Email_Classification_using_BERT_and_GPT_2_final_V2

January 16, 2026

1 Email spam classification with BERT and GPT-2(V2)

1.1 1.Download data

Download the data from Kaggle and unzip it.

The folder structure is set up for Google Colab; adjust it as needed for your environment.

```
[1]: import os

# Local path
filepath = "/content/emails-for-spam-or-ham-classification-trec-2007.zip"

# File original URL
url = "https://www.kaggle.com/api/v1/datasets/download/bayes2003/
      ↪emails-for-spam-or-ham-classification-trec-2007"

# Skip download & unzip if file existed
if os.path.exists(filepath):
    print(f"{filepath} already existed, skipping downloading.")
else:
    print(f"{filepath} not found, download and pre-process.")
    !curl -L -o {filepath} {url}
    !unzip {filepath}
```

/content/emails-for-spam-or-ham-classification-trec-2007.zip not found, download and pre-process.

% Total	% Received	% Xferd	Average Speed	Time	Time	Time	Current
			Dload	Upload	Total	Spent	Left
							Speed
0	0	0	0	0	0	--:--:--	0
100	482M	100	482M	0	20.0M	0:00:24	21.7M

Archive: /content/emails-for-spam-or-ham-classification-trec-2007.zip
inflating: email_origin.csv
inflating: email_text.csv
inflating: trec07p.tgz

1.2 2.Install missing python libs

```
[2]: import importlib.util
import sys
import subprocess

# Install all the libraries, skip it if already installed
def install_if_missing(package_name, import_name=None):
    if import_name is None:
        import_name = package_name
    if importlib.util.find_spec(import_name) is None:
        print(f"Installing {package_name}...")
        subprocess.check_call([sys.executable, "-m", "pip", "install", package_name])
    else:
        print(f"{package_name} is already installed.")

install_if_missing("matplotlib", "matplotlib")
install_if_missing("torch", "torch")
install_if_missing("scikit-learn", "sklearn")
install_if_missing("nltk", "nltk")
install_if_missing("torchviz", "torchviz")
install_if_missing("datasets", "datasets")
install_if_missing("pandas", "pandas")
install_if_missing("transformers", "transformers")
install_if_missing("sentence-transformers", "sentence_transformers")
```

matplotlib is already installed.
torch is already installed.
scikit-learn is already installed.
nltk is already installed.
Installing torchviz...
datasets is already installed.
pandas is already installed.
transformers is already installed.
sentence-transformers is already installed.

1.3 3.Import libs and functions

```
[3]: # Import all the libraries and related functions

import os
import glob
import re
import pandas as pd
import re
import string
import numpy as np
```

```

import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, TensorDataset
from torch.optim.lr_scheduler import StepLR

from sklearn import svm
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import accuracy_score, f1_score, precision_score, \
    recall_score, classification_report, \
    precision_recall_fscore_support, confusion_matrix

from sklearn.model_selection import train_test_split

import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

from transformers import BertModel, BertTokenizer
from transformers import GPT2Tokenizer, GPT2Model
from transformers import GPT2ForSequenceClassification
from transformers import Trainer
from transformers import TrainingArguments

from sentence_transformers import SentenceTransformer

```

1.4 4.Download common words

[4]: *# Download related common words which will be used in the pre-processing step*

```

nltk.download('wordnet')
nltk.download('stopwords')
nltk.download('punkt_tab')

```

```

[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.

```

[4]: True

1.5 5.Read email dataset

```
[5]: # Read csv file to memory
df = pd.read_csv(r'email_text.csv')

# Add new column 'category' based on 'label' to show spam and ham information
df['category'] = df['label'].apply(lambda x: 'spam' if x == 1 else 'ham')

# Show samples
df.head()
```

```
[5]:
```

	label	text	category
0	1	do you feel the pressure to perform and not ri...	spam
1	0	hi i've just updated from the gulus and i chec...	ham
2	1	mega authenticv i a g r a discount pricec i a ...	spam
3	1	hey billy it was really fun going out the othe...	spam
4	1	system of the home it will have the capabiliti...	spam

1.6 6.Show dataset information and percentage of ham and spam

Given the relatively small imbalance between ham and spam, we will not apply dataset balancing

```
[6]: # Show data frame basic information
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 53668 entries, 0 to 53667
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   label       53668 non-null  int64
1   text        53668 non-null  object
2   category    53668 non-null  object
dtypes: int64(1), object(2)
memory usage: 1.2+ MB
```

```
[7]: df.category.value_counts(normalize=True)
```

```
[7]: category
spam    0.557558
ham     0.442442
Name: proportion, dtype: float64
```

```
[8]: df.category.value_counts()
```

```
[8]: category
spam    29923
ham     23745
Name: count, dtype: int64
```

1.7 7. Define cleaner function, then pre-processing

```
[9]: def text_cleaner(text):

    # Convert to lower case
    text = text.lower()

    # Replace the abbreviation with the full form
    text = re.sub(r'\s+', ' ', text)
    text = re.sub(r"won't", "will not", text)
    text = re.sub(r"can't", "cannot", text)
    text = re.sub(r"n't", " not", text)
    text = re.sub(r" 've", " have", text)
    text = re.sub(r" 'll", " will", text)
    text = re.sub(r" 'd", " would", text)
    text = re.sub(r" 're", " are", text)
    text = re.sub(r" 's", " is", text)
    text = re.sub(r" 'm", " am", text)

    # Removes all punctuation and special symbols from text
    text = re.sub(r'[^w\s]', '', text)

    # Only english stop words will be processed
    stop_words = set(stopwords.words('english'))

    # Tokenize
    tokens = word_tokenize(text)

    # Remove all stop words from dataset
    filtered_text = [word for word in tokens if word.lower() not in stop_words]

    # Lemmatizer
    lemmatizer = WordNetLemmatizer()
    lemmatized_text = [lemmatizer.lemmatize(word) for word in filtered_text]

    cleaned_text = ' '.join(lemmatized_text)

    # Processed dataset ready for training
    return cleaned_text
```

```
[10]: # Pre processing dataset
df['cleaned_text'] = df['text'].apply(text_cleaner)
df.head()
```

```
[10]:   label          text category \
0      1  do you feel the pressure to perform and not ri...    spam
1      0  hi i've just updated from the gulus and i chec...    ham
```

```

2      1 mega authenticv i a g r a discount pricec i a ...      spam
3      1 hey billy it was really fun going out the othe...      spam
4      1 system of the home it will have the capabiliti...      spam

```

```

                                cleaned_text
0  feel pressure perform rising occasion try v ia...
1  hi updated gulu check mirror seems little typo...
2  mega authenticv g r discount pricec l discount...
3  hey billy really fun going night talking said ...
4  system home capability linked far know within ...

```

1.8 8.Split dataset

```

[11]: # split to 70%-15%-15%(Training-Validate-Test)

texts = df["cleaned_text"].tolist()
labels = df["label"].tolist()

X_temp, X_test, y_temp, y_test = train_test_split(
    texts, labels,
    test_size=0.15,
    random_state=42,
    stratify=labels
)

X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp,
    test_size = 0.1764706, # 15% total (0.15/0.85)
    random_state=42,
    stratify=y_temp
)

```

1.9 9.Base model to compare(google-bert/bert-base-uncased)

```

[12]: bert_model_name = 'google-bert/bert-base-uncased'
bert_model = SentenceTransformer(bert_model_name)
bert_tokenizer = BertTokenizer.from_pretrained(bert_model_name)
bert_epoch_num = 30

```

WARNING:sentence_transformers.SentenceTransformer:No sentence-transformers model found with name google-bert/bert-base-uncased. Creating a new one with mean pooling.

```

config.json: 0%|          | 0.00/570 [00:00<?, ?B/s]
model.safetensors: 0%|          | 0.00/440M [00:00<?, ?B/s]
tokenizer_config.json: 0%|          | 0.00/48.0 [00:00<?, ?B/s]

```

vocab.txt: 0.00B [00:00, ?B/s]

tokenizer.json: 0.00B [00:00, ?B/s]

1.9.1 9.1 Define SpamClassifier class

```
[13]: class SpamClassifier_bert(nn.Module):
    def __init__(self, input_size, hidden_size, output_size, dropout_prob=0.2):
        super().__init__()
        self.fc1 = nn.Linear(input_size, hidden_size)
        self.fc2 = nn.Linear(hidden_size, output_size)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(dropout_prob)

    def forward(self, x):
        out = self.fc1(x)
        out = self.relu(out)
        out = self.dropout(out)
        out = self.fc2(out)
        out = torch.sigmoid(out)
        return out
```

1.9.2 9.2.Encoding text to vectors

```
[14]: bert_X_train = bert_model.  

    ↪ encode(X_train, convert_to_numpy=True, show_progress_bar=True)  
  

bert_X_val = bert_model.encode(X_val, convert_to_numpy=True, ↪  

    ↪ show_progress_bar=True)  
  

bert_X_test = bert_model.encode(X_test, convert_to_numpy=True, ↪  

    ↪ show_progress_bar=True)  
  

bert_X_train_tensor = torch.tensor(bert_X_train)  

bert_X_val_tensor = torch.tensor(bert_X_val)  

bert_X_test_tensor = torch.tensor(bert_X_test)  
  

bert_y_train_tensor = torch.tensor(y_train, dtype=torch.float32)  

bert_y_val_tensor = torch.tensor(y_val, dtype=torch.float32)  

bert_y_test_tensor = torch.tensor(y_test, dtype=torch.float32)  
  

bert_train_dataset = TensorDataset(bert_X_train_tensor, bert_y_train_tensor)  

bert_val_dataset = TensorDataset(bert_X_val_tensor, bert_y_val_tensor)  

bert_test_dataset = TensorDataset(bert_X_test_tensor, bert_y_test_tensor)  
  

bert_train_loader = DataLoader(bert_train_dataset, batch_size=32)  

bert_val_loader = DataLoader(bert_val_dataset, batch_size=32)  

bert_test_loader = DataLoader(bert_test_dataset, batch_size=32)
```

Batches: 0%| | 0/1174 [00:00<?, ?it/s]

Batches: 0%| | 0/252 [00:00<?, ?it/s]

Batches: 0%| | 0/252 [00:00<?, ?it/s]

1.9.3 9.3. Define model parameters

```
[15]: bert_input_size = bert_X_train_tensor.shape[1]  

bert_hidden_size = 256  

bert_output_size = 1  

bert_criterion = nn.BCELoss()  

bert_spam_classifier_model = SpamClassifier_bert(bert_input_size, ↪  

    ↪ bert_hidden_size, bert_output_size)  
  

bert_optimizer = optim.Adam(bert_spam_classifier_model.parameters(), lr=1e-3)
```

1.9.4 9.4. Training

```
[16]: bert_train_losses = []  

bert_val_losses = []  
  

bert_val_accuracies = []
```



```

bert_val_precisions = []
bert_val_recalls = []
bert_val_f1s = []

bert_best_val_loss = float("inf")
bert_best_model_state = None
best_epoch = 0

for epoch in range(bert_epoch_num):

    bert_spam_classifier_model.train()
    running_loss = 0.0
    correct_train = 0
    total_train = 0
    for inputs, labels in bert_train_loader:
        labels = labels.unsqueeze(1).float()
        outputs = bert_spam_classifier_model(inputs)
        bert_optimizer.zero_grad()
        loss = bert_criterion(outputs, labels)
        loss.backward()
        bert_optimizer.step()
        running_loss += loss.item()

        predicted_train = (outputs > 0.5).float()
        correct_train += (predicted_train == labels).sum().item()
        total_train += labels.size(0)

    bert_train_losses.append(running_loss / len(bert_train_loader))

    bert_spam_classifier_model.eval()
    val_loss = 0.0
    correct_val = 0
    total_val = 0

    bert_val_preds = []
    bert_val_labels = []

    with torch.no_grad():
        for inputs, labels in bert_val_loader:
            labels = labels.unsqueeze(1).float()
            outputs = bert_spam_classifier_model(inputs)
            loss = bert_criterion(outputs, labels)
            val_loss += loss.item()

            predicted_val = (outputs > 0.5).float()
            correct_val += (predicted_val == labels).sum().item()
            total_val += labels.size(0)

```

```

bert_val_preds.extend(predicted_val.cpu().numpy())
bert_val_labels.extend(labels.cpu().numpy())

bert_val_losses.append(val_loss / len(bert_val_loader))

if bert_val_losses[-1] < bert_best_val_loss:
    bert_best_val_loss = bert_val_losses[-1]
    bert_best_model_state = bert_spam_classifier_model.state_dict()
    bert_best_epoch = epoch + 1

bert_val_accuracies.append(accuracy_score(bert_val_labels, bert_val_preds))
bert_val_precisions.append(precision_score(bert_val_labels, bert_val_preds))
bert_val_recalls.append(recall_score(bert_val_labels, bert_val_preds))
bert_val_f1s.append(f1_score(bert_val_labels, bert_val_preds))

print(f"Epoch {epoch+1}, Train Loss: {bert_train_losses[-1]}, Validate Loss:␣
↪{bert_val_losses[-1]}, Validate Accuracy: {bert_val_accuracies[-1]},␣
↪Validate Precision: {bert_val_precisions[-1]}, Validate Recall:␣
↪{bert_val_recalls[-1]}, Validate F1: {bert_val_f1s[-1]}")

```

```

Epoch 1, Train Loss: 0.12858441188610975, Validate Loss: 0.10092223463727841,
Validate Accuracy: 0.9669606260091914, Validate Precision: 0.9541836954183696,
Validate Recall: 0.9881933615504567, Validate F1: 0.970890785729919
Epoch 2, Train Loss: 0.07939986707531614, Validate Loss: 0.07416999336731221,
Validate Accuracy: 0.9759036144578314, Validate Precision: 0.971874313337728,
Validate Recall: 0.9852973936288706, Validate F1: 0.9785398230088496
Epoch 3, Train Loss: 0.06520162050229905, Validate Loss: 0.06356074579248559,
Validate Accuracy: 0.9806235250279468, Validate Precision: 0.9752138626891862,
Validate Recall: 0.990421029182446, Validate F1: 0.9827586206896551
Epoch 4, Train Loss: 0.056488766165186846, Validate Loss: 0.06046930248923008,
Validate Accuracy: 0.9822382312756179, Validate Precision: 0.9742470536883457,
Validate Recall: 0.9944308309200267, Validate F1: 0.984235475691765
Epoch 5, Train Loss: 0.04944571714988692, Validate Loss: 0.05818109074792278,
Validate Accuracy: 0.9831076884859024, Validate Precision: 0.9740797211936397,
Validate Recall: 0.9962129650256182, Validate F1: 0.9850220264317181
Epoch 6, Train Loss: 0.04347169450036999, Validate Loss: 0.05050119022341697,
Validate Accuracy: 0.9864613091541423, Validate Precision: 0.9811072056239016,
Validate Recall: 0.9948763644464246, Validate F1: 0.9879438115252738
Epoch 7, Train Loss: 0.04114138562767925, Validate Loss: 0.05137305897069237,
Validate Accuracy: 0.984473978387778, Validate Precision: 0.9804051078819903,
Validate Recall: 0.9919803965248385, Validate F1: 0.9861587864023917
Epoch 8, Train Loss: 0.03773212855310616, Validate Loss: 0.05373568297294523,
Validate Accuracy: 0.9857160601167557, Validate Precision: 0.9777195281782438,
Validate Recall: 0.9971040320784139, Validate F1: 0.9873166427704864
Epoch 9, Train Loss: 0.03398300824142291, Validate Loss: 0.04569434400510213,
Validate Accuracy: 0.9880760154018134, Validate Precision: 0.982641177763129,

```

Validate Recall: 0.9962129650256182, Validate F1: 0.9893805309734514
Epoch 10, Train Loss: 0.03214247360028976, Validate Loss: 0.047771125778949465,
Validate Accuracy: 0.9873307663644267, Validate Precision: 0.9811362140820356,
Validate Recall: 0.9964357317888171, Validate F1: 0.9887267904509284
Epoch 11, Train Loss: 0.029840914307384187, Validate Loss: 0.04854271062654406,
Validate Accuracy: 0.9870823500186312, Validate Precision: 0.9802847754654983,
Validate Recall: 0.9968812653152149, Validate F1: 0.9885133642588911
Epoch 12, Train Loss: 0.026739393819434475, Validate Loss: 0.04729128378306678,
Validate Accuracy: 0.9877033908831201, Validate Precision: 0.9817822651448639,
Validate Recall: 0.9964357317888171, Validate F1: 0.9890547263681592
Epoch 13, Train Loss: 0.025229795390616185, Validate Loss: 0.041138063486349505,
Validate Accuracy: 0.9886970562663023, Validate Precision: 0.9860742705570292,
Validate Recall: 0.99376253063043, Validate F1: 0.9899034727615666
Epoch 14, Train Loss: 0.02454586683834009, Validate Loss: 0.045805994537081535,
Validate Accuracy: 0.9879518072289156, Validate Precision: 0.982001755926251,
Validate Recall: 0.996658498552016, Validate F1: 0.9892758430071863
Epoch 15, Train Loss: 0.023447152180427373, Validate Loss: 0.04382613693312686,
Validate Accuracy: 0.9878275990560179, Validate Precision: 0.9824214458360799,
Validate Recall: 0.9959901982624193, Validate F1: 0.9891592920353982
Epoch 16, Train Loss: 0.020313344236172492, Validate Loss: 0.04805577492839398,
Validate Accuracy: 0.9870823500186312, Validate Precision: 0.981762250054933,
Validate Recall: 0.9953218979728224, Validate F1: 0.988495575221239
Epoch 17, Train Loss: 0.022867109610981876, Validate Loss: 0.04426077076941655,
Validate Accuracy: 0.9895665134765868, Validate Precision: 0.9860957845950121,
Validate Recall: 0.9953218979728224, Validate F1: 0.9906873614190688
Epoch 18, Train Loss: 0.0183319564405264, Validate Loss: 0.04848672823705812,
Validate Accuracy: 0.9878275990560179, Validate Precision: 0.9843370836090889,
Validate Recall: 0.9939852973936288, Validate F1: 0.9891376634892485
Epoch 19, Train Loss: 0.01830159588540045, Validate Loss: 0.048556723831285446,
Validate Accuracy: 0.9873307663644267, Validate Precision: 0.9873361475227728,
Validate Recall: 0.9899754956560481, Validate F1: 0.9886540600667408
Epoch 20, Train Loss: 0.017399208005319522, Validate Loss: 0.04582799866872688,
Validate Accuracy: 0.9893180971307912, Validate Precision: 0.9869497898694979,
Validate Recall: 0.9939852973936288, Validate F1: 0.9904550499445061
Epoch 21, Train Loss: 0.0170347151374087, Validate Loss: 0.04964728830111112,
Validate Accuracy: 0.9888212644392002, Validate Precision: 0.9854336790995365,
Validate Recall: 0.9946535976832257, Validate F1: 0.9900221729490022
Epoch 22, Train Loss: 0.016539009169220936, Validate Loss: 0.05893497459617229,
Validate Accuracy: 0.9889454726120979, Validate Precision: 0.9884547069271759,
Validate Recall: 0.9917576297616396, Validate F1: 0.9901034137662627
Epoch 23, Train Loss: 0.014195957535819344, Validate Loss: 0.05040267707248686,
Validate Accuracy: 0.9885728480934045, Validate Precision: 0.9865014383713211,
Validate Recall: 0.9930942303408331, Validate F1: 0.9897868561278863
Epoch 24, Train Loss: 0.019003179881491705, Validate Loss: 0.057643644878702445,
Validate Accuracy: 0.9893180971307912, Validate Precision: 0.9858750827631869,
Validate Recall: 0.9950991312096236, Validate F1: 0.9904656319290466
Epoch 25, Train Loss: 0.013884750412893903, Validate Loss: 0.05741040474022734,
Validate Accuracy: 0.9899391379952801, Validate Precision: 0.9899955535793686,

Validate Recall: 0.9919803965248385, Validate F1: 0.9909869811950596
 Epoch 26, Train Loss: 0.013511910356795902, Validate Loss: 0.05897625748775681,
 Validate Accuracy: 0.9904359706868712, Validate Precision: 0.9904401956425077,
 Validate Recall: 0.9924259300512364, Validate F1: 0.9914320685434517
 Epoch 27, Train Loss: 0.014087439181816696, Validate Loss: 0.06439862930787173,
 Validate Accuracy: 0.9875791827102224, Validate Precision: 0.9906103286384976,
 Validate Recall: 0.9870795277344621, Validate F1: 0.9888417763891988
 Epoch 28, Train Loss: 0.012826047420955564, Validate Loss: 0.09182003429889717,
 Validate Accuracy: 0.9879518072289156, Validate Precision: 0.9826373626373627,
 Validate Recall: 0.9959901982624193, Validate F1: 0.9892687244164178
 Epoch 29, Train Loss: 0.012923071920937881, Validate Loss: 0.07037364277651625,
 Validate Accuracy: 0.9899391379952801, Validate Precision: 0.9880425155004429,
 Validate Recall: 0.9939852973936288, Validate F1: 0.9910049972237646
 Epoch 30, Train Loss: 0.011509350053579843, Validate Loss: 0.061402200250822284,
 Validate Accuracy: 0.9893180971307912, Validate Precision: 0.9902026274771766,
 Validate Recall: 0.9906437959456449, Validate F1: 0.9904231625835189

```

[17]: # Find the index (epoch) with the lowest validate loss
#min_loss_idx = bert_val_losses.index(min(bert_val_losses))
#min_epoch = min_loss_idx + 1
#min_val_loss = bert_val_losses[min_loss_idx]
#corresponding_train_loss = bert_train_losses[min_loss_idx]
#corresponding_val_acc = bert_val_accuracies[min_loss_idx]
#corresponding_val_precision = bert_val_precisions[min_loss_idx]
#corresponding_val_recall = bert_val_recalls[min_loss_idx]
#corresponding_val_f1 = bert_val_f1s[min_loss_idx]

# Print the values
#print(f"Epoch with minimum validate loss: {min_epoch}")
#print(f"Train Loss: {corresponding_train_loss:.4f}")
#print(f"Validate Loss: {min_val_loss:.4f}")
#print(f"Validate Accuracy: {corresponding_val_acc:.4f}")
#print(f"Validate Precision: {corresponding_val_precision:.4f}")
#print(f"Validate Recall: {corresponding_val_recall:.4f}")
#print(f"Validate F1: {corresponding_val_f1:.4f}")
  
```

```

[18]: print(f"Epoch with minimum validate loss: {bert_best_epoch}")
      print(f"minimum validate loss:{bert_best_val_loss:.4f}")
  
```

Epoch with minimum validate loss: 13
 minimum validate loss:0.0411

1.9.5 9.5. Plot Losses and Accuracies

```

[20]: plt.figure(figsize=(14, 5))

# Loss plot
plt.subplot(1, 2, 1)
  
```

```

plt.plot(range(1, bert_epoch_num + 1), bert_train_losses, label='Training Loss')
plt.plot(range(1, bert_epoch_num + 1), bert_val_losses, label='Validate Loss')
plt.scatter(bert_best_epoch, bert_best_val_loss, color='red', label='Min Val_
↳Loss', zorder=5)
plt.annotate(f'{bert_best_epoch}',
            (bert_best_epoch, bert_best_val_loss),
            textcoords="offset points",
            xytext=(0,10),
            ha='center',
            color='blue',
            fontsize=9)
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Training and validate Losses')
plt.legend()

# Accuracy plot
plt.subplot(1, 2, 2)
plt.plot(range(1, bert_epoch_num + 1), bert_val_accuracies, label='Validate_
↳Accuracy')

plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('Validate Accuracies')
plt.legend()

plt.tight_layout()
plt.show()

```



```
[21]: bert_spam_classifier_model.load_state_dict(bert_best_model_state)
```

```
print(f"Loaded best model from epoch {bert_best_epoch} with val loss_␣  
↪{bert_best_val_loss:.4f}")
```

Loaded best model from epoch 13 with val loss 0.0411

```
[22]: bert_spam_classifier_model.eval()  
  
bert_test_preds = []  
bert_test_labels = []  
  
with torch.no_grad():  
    for inputs, labels in bert_test_loader:  
        labels = labels.unsqueeze(1).float()  
        outputs = bert_spam_classifier_model(inputs)  
        predicted = (outputs > 0.5).float()  
  
        bert_test_preds.extend(predicted.cpu().numpy())  
        bert_test_labels.extend(labels.cpu().numpy())  
  
bert_test_preds = np.array(bert_test_preds).flatten()  
bert_test_labels = np.array(bert_test_labels).flatten()
```

```
[23]: bert_cm = confusion_matrix(bert_test_labels, bert_test_preds)  
print("Confusion Matrix:")  
print(bert_cm)  
  
bert_test_accuracy = accuracy_score(bert_test_labels, bert_test_preds)  
bert_test_precision = precision_score(bert_test_labels, bert_test_preds)  
bert_test_recall = recall_score(bert_test_labels, bert_test_preds)  
bert_test_f1 = f1_score(bert_test_labels, bert_test_preds)  
  
print("Test Accuracy :", bert_test_accuracy)  
print("Test Precision:", bert_test_precision)  
print("Test Recall   :", bert_test_recall)  
print("Test F1-score  :", bert_test_f1)
```

Confusion Matrix:

```
[[3525  37]  
 [ 58 4431]]
```

Test Accuracy : 0.9882002235747113

Test Precision: 0.9917188898836168

Test Recall : 0.9870795277344621

Test F1-score : 0.98939377023557

1.10 10.Model to compare(GPT-2)

1.10.1 10.1 Define SpamClassifierGPT2 class

```
[25]: class SpamClassifierGPT2(torch.utils.data.Dataset):
    def __init__(self, texts, labels, tokenizer, max_length=128):
        self.encodings = tokenizer(
            texts,
            padding="max_length",
            truncation=True,
            max_length=max_length)
        self.labels = labels

    def __getitem__(self, idx):
        item = {key: torch.tensor(val[idx]) for key, val in self.encodings.
            items()}
        item["labels"] = torch.tensor(self.labels[idx], dtype=torch.long)
        return item

    def __len__(self):
        return len(self.labels)
```

1.10.2 10.2 Setup models

```
[26]: gpt2_tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
gpt2_tokenizer.pad_token = gpt2_tokenizer.eos_token

gpt2_model = GPT2ForSequenceClassification.from_pretrained(
    "gpt2",
    num_labels = 2 # Binary classification
)
gpt2_model.config.pad_token_id = gpt2_tokenizer.pad_token_id

gpt2_epoch_num = 30
```

```
tokenizer_config.json: 0%|          | 0.00/26.0 [00:00<?, ?B/s]
vocab.json: 0%|          | 0.00/1.04M [00:00<?, ?B/s]
merges.txt: 0%|          | 0.00/456k [00:00<?, ?B/s]
tokenizer.json: 0%|          | 0.00/1.36M [00:00<?, ?B/s]
config.json: 0%|          | 0.00/665 [00:00<?, ?B/s]
model.safetensors: 0%|          | 0.00/548M [00:00<?, ?B/s]
```

Some weights of GPT2ForSequenceClassification were not initialized from the model checkpoint at gpt2 and are newly initialized: ['score.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

1.10.3 10.3 Setup datasets

```
[27]: # setup datasets

gpt2_train_dataset = SpamClassifierGPT2(X_train, y_train, gpt2_tokenizer)
gpt2_val_dataset   = SpamClassifierGPT2(X_val, y_val, gpt2_tokenizer)
gpt2_test_dataset  = SpamClassifierGPT2(X_test, y_test, gpt2_tokenizer)
```

1.10.4 10.4 define extra functions

```
[28]: def compute_metrics(pred):
    preds = np.argmax(pred.predictions, axis=1)
    labels = pred.label_ids
    acc = accuracy_score(labels, preds)
    precision, recall, f1, _ = precision_recall_fscore_support(labels, preds,
↪average="binary")
    return {
        "accuracy": acc,
        "precision": precision,
        "recall": recall,
        "f1": f1
    }
```

1.10.5 10.4 Training

```
[29]: training_args = TrainingArguments(
    output_dir = "./gpt2-spam",
    eval_strategy = "epoch",
    save_strategy = "epoch",
    logging_strategy = "epoch",
    per_device_train_batch_size = 64,
    per_device_eval_batch_size = 64,
    num_train_epochs = gpt2_epoch_num,
    weight_decay = 0.01,
    logging_dir = "./logs",
    load_best_model_at_end = True,
    metric_for_best_model="eval_loss",
    gradient_accumulation_steps = 2,
    report_to = "none",
    fp16 = True
)

gpt2_trainer = Trainer(
    model = gpt2_model,
    args = training_args,
    train_dataset = gpt2_train_dataset,
    eval_dataset = gpt2_val_dataset,
```



```

        processing_class = gpt2_tokenizer,
        compute_metrics = compute_metrics
    )

gpt2_trainer.train()

```

<IPython.core.display.HTML object>

```

[29]: TrainOutput(global_step=8820, training_loss=0.00847261143244313,
  metrics={'train_runtime': 1878.5087, 'train_samples_per_second': 599.933,
  'train_steps_per_second': 4.695, 'total_flos': 7.361905299554304e+16,
  'train_loss': 0.00847261143244313, 'epoch': 30.0})

```

1.10.6 10.5. Plot Losses and Accuracies

```

[30]: gpt2_logs = gpt2_trainer.state.log_history

# Convert to DataFrame
gpt2_log_df = pd.DataFrame(gpt2_logs)
# Filter for loss and epoch info
gpt2_loss_df = gpt2_log_df[['epoch', 'loss', 'eval_loss', 'eval_accuracy']].
    dropna(how='all')

# Group by epoch and aggregate by 'first non-null'
gpt2_merged_df = gpt2_log_df.groupby('epoch').agg({
    'loss': 'first',
    'eval_loss': 'first',
    'eval_accuracy': 'first'
}).reset_index()

# Show merged result
print(gpt2_merged_df)

```

	epoch	loss	eval_loss	eval_accuracy
0	1.0	0.1481	0.049595	0.985716
1	2.0	0.0316	0.036898	0.988821
2	3.0	0.0174	0.031551	0.990684
3	4.0	0.0093	0.038837	0.989442
4	5.0	0.0058	0.041759	0.991057
5	6.0	0.0039	0.046312	0.990809
6	7.0	0.0034	0.057170	0.989318
7	8.0	0.0032	0.116469	0.980375
8	9.0	0.0030	0.061354	0.989691
9	10.0	0.0025	0.055115	0.991802
10	11.0	0.0024	0.097905	0.984722
11	12.0	0.0024	0.067021	0.991554
12	13.0	0.0019	0.064656	0.990809
13	14.0	0.0016	0.071269	0.990560

14	15.0	0.0020	0.090240	0.988200
15	16.0	0.0016	0.055435	0.991554
16	17.0	0.0014	0.059709	0.991926
17	18.0	0.0010	0.068933	0.991430
18	19.0	0.0010	0.078548	0.990063
19	20.0	0.0010	0.080103	0.988945
20	21.0	0.0009	0.062454	0.992423
21	22.0	0.0014	0.075878	0.992423
22	23.0	0.0011	0.079748	0.990560
23	24.0	0.0017	0.101087	0.987579
24	25.0	0.0008	0.064162	0.992672
25	26.0	0.0008	0.069256	0.992299
26	27.0	0.0008	0.074577	0.991554
27	28.0	0.0007	0.076842	0.991678
28	29.0	0.0007	0.076010	0.991430
29	30.0	0.0007	0.075791	0.991430

```
[31]: # Find the row with minimum eval_loss
gpt2_min_eval_loss_row = gpt2_merged_df.loc[gpt2_merged_df['eval_loss'].
    ↪idxmin()]

gpt2_min_epoch = gpt2_min_eval_loss_row['epoch']
gpt2_min_eval_loss = gpt2_min_eval_loss_row['eval_loss']
gpt2_min_eval_accuracy = gpt2_min_eval_loss_row['eval_accuracy']

print(f"Epoch with minimum eval_loss: {gpt2_min_epoch}")
print(f"Minimum eval_loss: {gpt2_min_eval_loss}")
```

Epoch with minimum eval_loss: 3.0
Minimum eval_loss: 0.03155067190527916

```
[32]: plt.figure(figsize=(14, 5))

# Loss plot
plt.subplot(1, 2, 1)
plt.plot(range(1, gpt2_epoch_num + 1), gpt2_merged_df['loss'], label='Training_
    ↪Loss')
plt.plot(range(1, gpt2_epoch_num + 1), gpt2_merged_df['eval_loss'], label='Eval_
    ↪Loss')
plt.scatter(gpt2_min_epoch, gpt2_min_eval_loss, color='red', label='Min Eval_
    ↪Loss', zorder=5)

plt.annotate(
    f'{int(gpt2_min_epoch)}',
    (gpt2_min_epoch, gpt2_min_eval_loss),
    textcoords="offset points",
    xytext=(0, 10),
    ha='center',
```

```

        color='blue',
        fontsize=9
    )

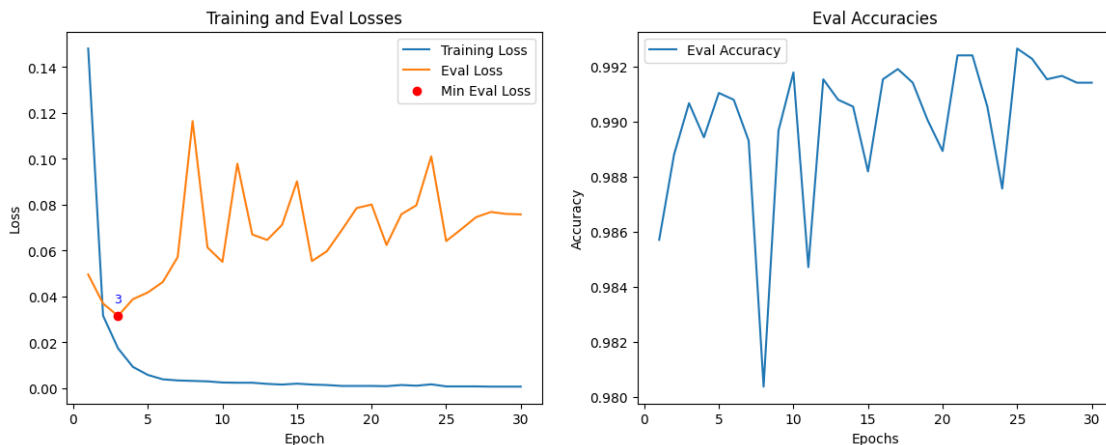
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training and Eval Losses")
plt.legend()

# Accuracy plot
plt.subplot(1, 2, 2)
plt.plot(range(1, gpt2_epoch_num + 1), gpt2_merged_df['eval_accuracy'],
        label='Eval Accuracy')

plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('Eval Accuracies')
plt.legend()

plt.show()

```



```

[33]: # since we set load_best_model_at_end = True and
        metric_for_best_model="eval_loss", so the default model is the best model we
        found

gpt2_test_metrics = gpt2_trainer.evaluate(gpt2_test_dataset)

print("Final TEST results (GPT-2):")
print(f"Accuracy : {gpt2_test_metrics['eval_accuracy']:.4f}")

```

```
print(f"Precision: {gpt2_test_metrics['eval_precision']:.4f}")
print(f"Recall    : {gpt2_test_metrics['eval_recall']:.4f}")
print(f"F1-score  : {gpt2_test_metrics['eval_f1']:.4f}")
```

<IPython.core.display.HTML object>

Final TEST results (GPT-2):
 Accuracy : 0.9914
 Precision: 0.9887
 Recall : 0.9960
 F1-score : 0.9923

```
[35]: gpt2_test_output = gpt2_trainer.predict(gpt2_test_dataset)
      # Logits -> predicted class
      gpt2_test_logits = gpt2_test_output.predictions
      gpt2_test_preds = np.argmax(gpt2_test_logits, axis=1)

      # True labels
      gpt2_test_labels = gpt2_test_output.label_ids
```

<IPython.core.display.HTML object>

```
[36]: gpt2_cm = confusion_matrix(gpt2_test_labels, gpt2_test_preds)
      print("Confusion Matrix:")
      print(gpt2_cm)

      gpt2_test_accuracy = accuracy_score(gpt2_test_labels, gpt2_test_preds)
      gpt2_test_precision = precision_score(gpt2_test_labels, gpt2_test_preds)
      gpt2_test_recall    = recall_score(gpt2_test_labels, gpt2_test_preds)
      gpt2_test_f1        = f1_score(gpt2_test_labels, gpt2_test_preds)

      print("Test Accuracy :", gpt2_test_accuracy)
      print("Test Precision:", gpt2_test_precision)
      print("Test Recall   :", gpt2_test_recall)
      print("Test F1-score :", gpt2_test_f1)
```

Confusion Matrix:
 [[3511 51]
 [18 4471]]
 Test Accuracy : 0.9914296360700534
 Test Precision: 0.9887218045112782
 Test Recall : 0.9959901982624193
 Test F1-score : 0.9923426922650095